Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Объектно-ориентированное программирование

К защите допустить
И.О. Заведующего кафедрой
информатики
_____ С.И. Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

ПЛАТФОРМА ДЛЯ УПРАВЛЕНИЯ КОЛЛЕКЦИЯМИ ФРАКТАЛОВ

БГУИР КП 6-05-0612-02 113 ПЗ

Студент Н.А. Халамов

Руководитель А.Г. Бокун

Минск 2026

СОДЕРЖАНИЕ

Введение 5
1 Анализ существующих аналогов 7
1.1 Десктопные приложения для генерации фракталов 7
1.2 Веб-ориентированные инструменты 9
1.3 Системы управления цифровыми коллекциями 10
1.4 Выявленные проблемы и обоснование актуальности проекта 10
2 Формирование функциональных требований 12
2.1 Общие принципы формирования требований 12
2.2 Требования к генерации фракталов 12
2.3 Требования к управлению коллекцией 13
2.4 Требования к веб-интерфейсу и программному интерфейсу 14
2.5 Требования к хранению данных 14
2.6 Требования к безопасности и надёжности 15
2.7 Обоснование выбора требований 15
3 Анализ ООП в литературе 16
3.1 Значимость объектно-ориентированного проектирования 16
3.2 Фундаментальные принципы ООП 17
3.3 Принципы чистой архитектуры и управление зависимостями 18
3.4 Применение паттернов проектирования 18
3.5 Особенности реализации ООП в языке Python 21
3.6 Архитектурные решения для разрабатываемой платформы 21
4 Проектирование системы 23
4.1 Выбор технологического стека 23
4.2 Архитектурное проектирование системы 24
4.3 Применение паттернов проектирования 25
4.4 Теоретические основы генерации фракталов 27
4.5 Реализация алгоритмов генерации 28
4.6 Проектирование системы хранения данных 29
4.7 Обеспечение асинхронности и производительности 30
5 Тестирование программного обеспечения 31
5.1 Методика и среда проведения тестирования 31
5.2 Тестирование механизмов аутентификации и авторизации 31
5.3 Проверка интерфейса генерации и валидации параметров 32
5.4 Тестирование отображения результатов 33
5.5 Тестирование управления коллекцией 34
5.6 Проверка удаления записей и расчёта статистики 36
5.7 Тестирование обработки ошибок 36
5.8 Оценка производительности 37
Заключение 38
Список использованных источников 40

ВВЕДЕНИЕ

В современном программировании визуализация математических моделей и управление цифровыми коллекциями представляют собой востребованные направления разработки программного обеспечения. Фрактальная графика, обладающая высокой вычислительной сложностью и значительной эстетической ценностью, находит применение в научных исследованиях, компьютерной графике, цифровом искусстве и образовательных технологиях. Однако эффективная работа с фрактальными изображениями требует специализированных инструментов, способных не только выполнять ресурсоёмкие вычисления, но и обеспечивать удобное хранение, каталогизацию и повторное использование результатов генерации.

Существующие программные решения предоставляют широкие возможности для генерации и визуализации, однако характеризуются высоким порогом вхождения для неподготовленных пользователей, отсутствием встроенных механизмов управления коллекциями и необходимостью локальной установки программного обеспечения. Веб-ориентированные генераторы, работающие непосредственно в браузере, обеспечивают доступность с любого устройства, но ограничены в вычислительных возможностях и не поддерживают сохранение параметров генерации на сервере. Универсальные системы управления цифровыми активами, в свою очередь, не адаптированы под специфику фрактальной графики и не предоставляют инструментов для параметрической генерации изображений.

Перенос логики генерации фракталов в веб-среду позволяет устранить многие из перечисленных ограничений. Веб-платформа, доступная через стандартный браузер, не требует установки дополнительного программного обеспечения, обеспечивает кроссплатформенность и возможность централизованного хранения данных. При этом современная архитектура веб-приложений, основанная на асинхронной обработке запросов и строгой типизации данных, позволяет эффективно распределять вычислительную нагрузку и обеспечивать отзывчивый пользовательский интерфейс даже при выполнении ресурсоёмких операций.

Разработка системы, поддерживающей различные типы фракталов и предусматривающей возможность расширения функциональности, требует применения гибкой и масштабируемой архитектуры. Без использования принципов объектно-ориентированного программирования кодовая база быстро становится неподдерживаемой при добавлении новых алгоритмов генерации, типов параметров или способов визуализации. Объектно-ориентированный подход позволяет выделить общие свойства различных типов фракталов, построить иерархию классов с чётко определёнными интерфейсами и обеспечить возможность полиморфного использования компонентов системы.

В рамках данной курсовой работы применяется объектно-ориентированный подход к проектированию архитектуры приложения. Для

реализации серверной части выбран фреймворк FastAPI, который обеспечивает высокую производительность за счёт асинхронной обработки запросов, автоматическую генерацию документации в формате OpenAPI и строгую типизацию данных через аннотации типов Python и библиотеку Pydantic. Также в работе применяется подход рефакторинга: существующая консольная логика генерации фракталов перерабатывается и инкапсулируется в классы, что позволяет отделить бизнес-логику вычислений от уровня представления и транспортировки данных.

Объектом исследования является процесс разработки программного комплекса для генерации, кастомизации и управления коллекциями фрактальных изображений. Предметом исследования выступают методы и средства объектно-ориентированного программирования на языке Python, а также архитектурные паттерны веб-приложений, применяемые для создания масштабируемой платформы на базе фреймворка FastAPI.

Целью курсовой работы является разработка веб-платформы для управления коллекциями фракталов, которая обеспечивает пользователю возможность генерации изображений с настраиваемыми параметрами, их сохранения и просмотра, при этом внутренняя архитектура приложения должна строго соответствовать принципам объектно-ориентированного программирования для обеспечения расширяемости и поддерживаемости кода.

Для реализации поставленной цели необходимо решить следующие задачи. Во-первых, выделить общие свойства различных типов фракталов для построения иерархии классов, обеспечивающей полиморфное использование алгоритмов генерации. Во-вторых, спроектировать объектную модель предметной области, реализовав базовые классы и интерфейсы для генерации изображений, валидации параметров и пост-обработки результатов. В-третьих, разработать REST API на фреймворке FastAPI для обработки запросов на генерацию, получение параметров и управление коллекцией сохранённых фракталов. В-четвёртых, реализовать механизм хранения данных фракталов и самих изображений в базе данных с использованием объектно-реляционного отображения. В-пятых, провести тестирование работоспособности платформы, проверить корректность работы методов и производительность программного интерфейса.

Практическая значимость работы заключается в создании функционального прототипа веб-платформы, который может быть использован как в образовательных целях для демонстрации принципов объектно-ориентированного проектирования, так и в качестве основы для дальнейшего развития в направлении полноценного сервиса для работы с фрактальной графикой.

1 АНАЛИЗ СУЩЕСТВУЮЩИХ АНАЛОГОВ

В рамках разработки платформы для управления коллекциями фракталов был проведён анализ существующих программных решений в области генерации и визуализации фрактальной графики. Цель анализа заключалась в выявлении функциональных возможностей, архитектурных особенностей и ограничений аналогов, чтобы обосновать актуальность и конкурентные преимущества разрабатываемой веб-платформы на базе фреймворка FastAPI. Анализ охватывает три категории программных продуктов: десктопные приложения для генерации фракталов, веб-ориентированные инструменты с возможностью работы в браузере, а также системы управления цифровыми коллекциями с элементами визуализации.

1.1 Десктопные приложения для генерации фракталов

Десктопные приложения для генерации фракталов представляют собой наиболее зрелую категорию программных решений. Ultra Fractal является коммерческим программным обеспечением для операционных систем Windows и macOS, ориентированным на создание фрактального искусства с профессиональным качеством визуализации. Внешний вид программы показан в соответствии с рисунком 1.1.

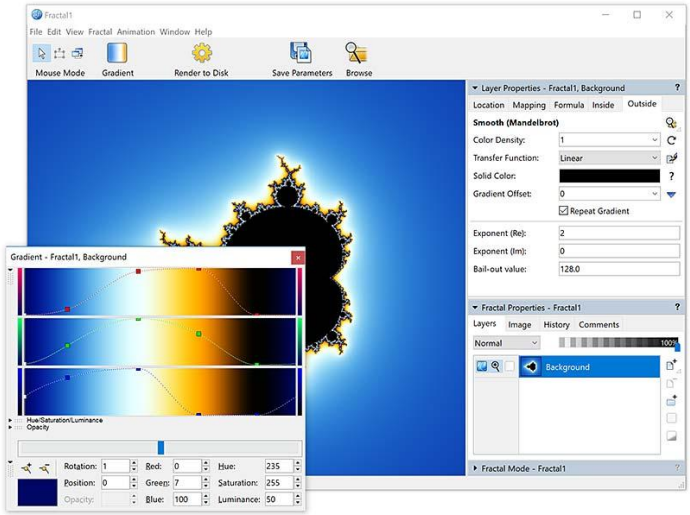


Рисунок 1.1 – Интерфейс программы UltraFractal

Функциональные возможности приложения включают более тысячи встроенных формул фракталов и алгоритмов раскраски, поддержку многослойной композиции с различными режимами наложения, возможность выполнения глубокого зумирования с произвольной точностью, а также генерацию анимаций и рендеринг в высоком разрешении. Расширяемость обеспечивается через пользовательские формулы на встроенном скриптовом языке. К преимуществам Ultra Fractal относятся интуитивный графический

интерфейс, не требующий глубокой математической подготовки пользователя, высокое качество рендеринга с поддержкой градиентов, активное сообщество разработчиков и библиотека пользовательских формул, а также подробная документация и обучающие материалы. Однако приложение имеет и существенные недостатки: платная лицензия с ограничениями в бесплатной версии, отсутствие веб-интерфейса и необходимость локальной установки, закрытый исходный код, ограничивающий возможность кастомизации, а также отсутствие встроенных механизмов управления коллекциями и совместной работы. Таким образом, Ultra Fractal представляет собой мощный инструмент для художественной визуализации, но не решает задачу централизованного управления коллекциями фракталов и не поддерживает веб-доступ.

Mandelbulb 3D представляет собой бесплатное приложение с открытым исходным кодом, предназначенное для генерации трёхмерных фракталов на основе нелинейных уравнений. Функциональные возможности включают поддержку десятков типов трёхмерных фракталов, таких как Mandelbulb, Mandelbox и JuliaBulb, расширенные настройки освещения, теней, глубины резкости и пост-обработки, генерацию анимаций и последовательностей кадров, а также экспорт результатов в распространённые графические форматы. Преимущества приложения заключаются в полной бесплатности и открытом исходном коде, высокой гибкости настройки параметров рендеринга, активном сообществе пользователей и доступности обучающих материалов. К недостаткам относятся ограниченная кроссплатформенность, поскольку приложение работает только под операционной системой Windows, отсутствие нативной поддержки других платформ, отсутствие встроенной системы хранения и каталогизации результатов, высокие требования к вычислительным ресурсам при рендеринге сложных сцен, а также отсутствие веб-интерфейса и программного интерфейса для интеграции с другими системами. Следовательно, Mandelbulb 3D является отличным инструментом для трёхмерной визуализации, но не подходит для задач управления коллекциями и не предоставляет возможностей для веб-развёртывания.

Apophysis и его кроссплатформенный наследник JWildfire представляют собой специализированные редакторы фрактальных пламен, основанные на алгоритмах итерированных функциональных систем. Функциональные возможности включают визуальное редактирование параметров аффинных преобразований и вариационных функций, поддержку анимации и рендеринга в высоком разрешении, экспорт в графические форматы, а также в формат параметров для последующего воспроизведения. В случае JWildfire обеспечивается кроссплатформенность благодаря использованию языка программирования Java. Преимущества данных решений заключаются в бесплатности и открытом исходном коде, глубокой кастомизации алгоритмов генерации, поддержке скриптов и плагинов для расширения функциональности, а также в активном сообществе и возможности обмена параметрами фракталов. Однако имеются и недостатки: сложный интерфейс с

высоким порогом вхождения для неподготовленных пользователей, отсутствие встроенных механизмов управления коллекциями, отсутствие веб-интерфейса и возможности удалённого доступа, а также значительные временные и вычислительные затраты при рендеринге больших изображений. Таким образом, Apophysis и JWildfire предоставляют мощные средства для генерации фрактальных пламен, но не решают задачу удобного хранения, поиска и совместного использования результатов.

FractalNow и его вариант с графическим интерфейсом QFractalNow представляют собой набор инструментов для быстрой генерации классических фракталов, таких как множество Мандельброта, множества Жюлиа и Multibrot. Функциональные возможности включают поддержку множества формул и методов раскраски, многопоточную обработку для ускорения вычислений, кэширование для повышения качества, а также сохранение и загрузку конфигураций генерации. Преимущества заключаются в высокой производительности благодаря оптимизированным алгоритмам, кроссплатформенности, возможности использования в скриптах для пакетной генерации и открытом исходном коде. К недостаткам относятся ограниченный набор типов фракталов по сравнению с узкоспециализированными инструментами, отсутствие веб-интерфейса и механизмов совместной работы, минималистичный интерфейс, не ориентированный на художественную работу, а также отсутствие встроенной базы данных для управления коллекциями. Следовательно, FractalNow эффективен для технической генерации фракталов, но не предоставляет возможностей для управления цифровыми коллекциями и веб-доступа. Внешний вид программы показан в соответствии с рисунком 1.2.

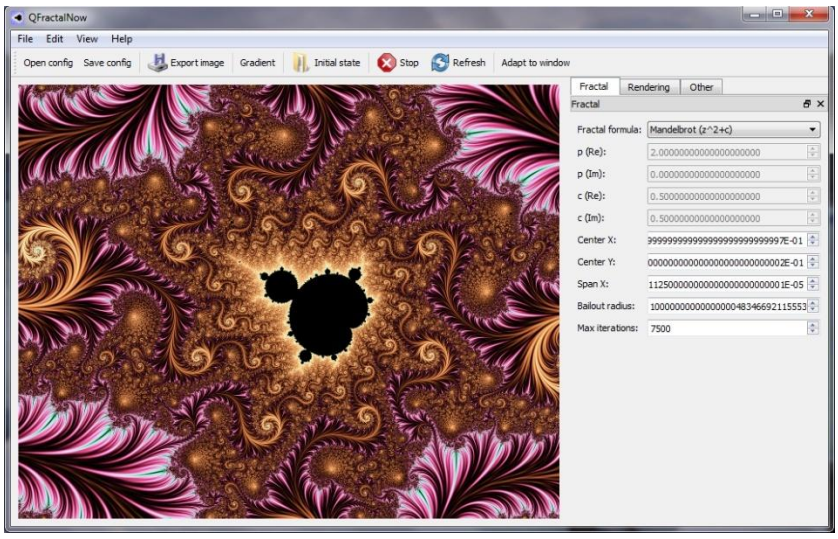


Рисунок 1.2 – Интерфейс программы FractalNow

1.2 Веб-ориентированные инструменты

Веб-ориентированные инструменты представляют собой

альтернативную категорию решений, работающих непосредственно в браузере. Онлайн-генераторы на базе технологий WebGL, такие как Fractal Foundation Tools, IceFractal и Fractal Art App, не требуют установки программного обеспечения и обеспечивают доступ с любого устройства с современным браузером. Преимущества включают интерактивную визуализацию в реальном времени и простоту использования для базовых сценариев. Однако такие решения имеют ограниченную вычислительную мощность по сравнению с десктопными приложениями, отсутствие сохранения параметров и результатов на сервере, отсутствие механизмов управления коллекциями и пользовательскими настройками, а также зависимость от качества интернет-соединения и возможностей браузера. Таким образом, веб-генераторы удобны для демонстрации и обучения, но не подходят для профессиональной работы и управления большими коллекциями.

FractalAI представляет собой экспериментальный веб-генератор фракталов, написанный с использованием технологий WebGL и regl, с акцентом на интерактивность. Преимущества включают современный стек технологий, интерактивное управление параметрами в реальном времени и открытый исходный код. Однако проект имеет ограниченный набор типов фракталов, отсутствие серверной части для хранения данных, отсутствие системы аутентификации и управления пользовательскими коллекциями, а также находится в стадии прототипа без активной поддержки. Следовательно, FractalAI демонстрирует потенциал веб-технологий для фрактальной визуализации, но не реализует полноценную платформу для управления коллекциями.

1.3 Системы управления цифровыми коллекциями

Системы управления цифровыми коллекциями представляют собой третью категорию анализируемых решений. Специализированные системы управления цифровыми активами, такие как Fracttal One, ориентированы на управление техническими активами и техническим обслуживанием, а не на фрактальную графику. Недостатки для задачи проекта включают отсутствие специализированных метаданных для фракталов, таких как параметры генерации, алгоритмы и трансформации, отсутствие интеграции с движками генерации фракталов, высокую стоимость и сложность развёртывания, а также избыточный функционал для задачи управления коллекциями фракталов. Таким образом, универсальные системы управления цифровыми активами не адаптированы под специфику фрактальной графики и не предоставляют инструментов для параметрической генерации.

1.4 Выявленные проблемы и обоснование актуальности проекта

Проведённый анализ выявил ряд неудовлетворённых потребностей

пользователей. Отсутствуют веб-платформы с полноценным управлением коллекциями: существующие десктопные решения не предоставляют возможности централизованного хранения и использования фракталов, а веб-генераторы не сохраняют параметры генерации и не поддерживают каталогизацию. Наблюдается сложность интеграции генерации в веб-приложения: большинство инструментов либо не имеют программного интерфейса, либо предоставляют только консольный интерфейс, что затрудняет их использование в современных веб-архитектурах. Отсутствует расширяемая объектно-ориентированная архитектура: многие проекты написаны процедурно или с минимальным использованием принципов объектно-ориентированного программирования, что усложняет добавление новых типов фракталов и алгоритмов. Также отмечается недостаточная типизация и документирование параметров: в существующих решениях параметры генерации часто хранятся в неструктурированном виде, что затрудняет воспроизведение результатов и обмен настройками.

Разрабатываемая платформа направлена на устранение указанных пробелов за счёт применения веб-архитектуры на базе фреймворка FastAPI с асинхронной обработкой запросов и строгой типизацией данных. Объектно-ориентированная модель предметной области позволяет легко добавлять новые типы фракталов через механизмы наследования и полиморфизма. Встроенная система хранения обеспечивает возможность сохранения как изображений, так и параметров генерации в структурированном виде. Программный интерфейс в стиле REST обеспечивает интеграцию с внешними системами и фронтенд-приложениями. Низкий порог вхождения достигается благодаря веб-интерфейсу, не требующему локальной установки программного обеспечения. Таким образом, проект сочетает в себе преимущества десктопных генераторов, такие как гибкость и производительность, с преимуществами веб-приложений, такими как доступность и масштабируемость, заполняя существующую нишу на рынке инструментов для работы с фрактальной графикой.

2 ФОРМИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

На основе анализа существующих аналогов, представленного в первом разделе, были сформулированы функциональные требования к разрабатываемой веб-платформе для управления коллекциями фракталов. Основной акцент при формировании требований сделан на объектно-ориентированную архитектуру системы, простоту использования пользователем.

2.1 Общие принципы формирования требований

Реалистичность реализации предполагает, что каждый функциональный пункт должен быть выполним с использованием стандартных инструментов языка программирования Python и распространённых библиотек с открытым исходным кодом. Данный принцип обеспечивает возможность завершения разработки в установленные сроки и минимизирует риски, связанные с зависимостью от непроверенных или экспериментальных решений. Соответствие объектно-ориентированной парадигме требует, чтобы архитектура системы демонстрировала применение классов, наследования, инкапсуляции и полиморфизма. Это означает, что бизнес-логика генерации фракталов должна быть выделена в отдельные классы, связанные через чётко определённые интерфейсы, а взаимодействие между компонентами должно осуществляться через механизмы внедрения зависимостей. Практическая полезность предполагает, что система должна решать конкретную задачу генерации, визуализации и хранения фрактальных изображений, а не представлять собой абстрактный учебный пример. Платформа должна предоставлять пользователю интуитивно понятный интерфейс для настройки параметров генерации, просмотра результатов и управления личной коллекцией.

Функциональные требования разделены на четыре логические группы: требования к генерации фракталов, требования к управлению коллекцией, требования к веб-интерфейсу и программному интерфейсу, а также требования к хранению данных. Такое структурирование позволяет последовательно реализовывать систему модуль за модулем и проводить независимое тестирование каждого компонента.

2.2 Требования к генерации фракталов

Система должна предоставлять пользователю возможность генерации фрактальных изображений четырёх типов: классическое множество Мандельброта, множество Жюлиа, фрактал «Корабль» и фрактальное пламя на основе системы итерированных функций. Каждый тип фрактала реализуется в виде отдельного класса, наследующего общий абстрактный интерфейс генератора, что обеспечивает полиморфное использование

алгоритмов и возможность расширения системы без модификации существующего кода.

Все параметры передаются в виде структурированного JSON-объекта, что обеспечивает удобство взаимодействия с веб-интерфейсом и упрощает валидацию входных данных. Система должна выполнять проверку корректности параметров перед запуском генерации: ширина и высота изображения должны быть положительными целыми числами, количество итераций – положительным значением, коэффициент гамма-коррекции – положительным числом. При обнаружении ошибок валидации пользователю возвращается информативное сообщение с указанием проблемного параметра.

Процесс генерации должен выполняться в асинхронном режиме, чтобы не блокировать обработку других запросов к серверу. Пользователь отправляет запрос на генерацию и получает уникальный идентификатор задачи, по которому может впоследствии проверить статус выполнения. Для реализации асинхронной обработки используется механизм пула потоков, позволяющий выполнять вычислительно сложные операции в фоновом режиме без потери отзывчивости веб-интерфейса.

2.3 Требования к управлению коллекцией

Каждый сгенерированный фрактал должен сохраняться в базе данных с набором метаданных, обеспечивающих возможность последующего поиска, фильтрации и воспроизведения. Минимальный набор полей включает: уникальный идентификатор в формате UUID, дату и время создания, тип фрактала, параметры генерации в формате JSON, путь к файлу изображения, размеры изображения в пикселях, время, затраченное на генерацию, и список тегов для категоризации. Такая структура данных позволяет реализовать гибкие механизмы фильтрации и группировки фракталов по различным критериям.

Пользователь системы должен иметь возможность просматривать список сохранённых фракталов с краткой информацией о каждом элементе коллекции. При выборе конкретного фрактала отображаются его полные параметры, превью изображения и метаданные, включая время генерации и размер файла. Должна быть реализована функция удаления фрактала из коллекции, которая удаляет как запись из базы данных, так и соответствующий файл изображения, обеспечивая целостность хранилища.

Для удобства организации коллекции система поддерживает механизм тегов, позволяющий пользователю присваивать фракталам произвольные метки для последующей фильтрации. Особый статус имеет тег «избранное», который визуально выделяет помеченные фракталы в интерфейсе. Пользователь может добавлять и удалять теги, а также фильтровать коллекцию по одному или нескольким тегам одновременно. Реализация механизма тегов основана на паттерне Composite, что обеспечивает единый интерфейс для работы с одиночными и групповыми элементами.

Для удобства обмена параметрами система поддерживает экспорт и импорт настроек генерации в формате JSON. Пользователь может выгрузить параметры понравившегося фрактала и передать их другому пользователю или использовать для воспроизведения результата на другой установке. При импорте параметров система автоматически проверяет их корректность, создаёт новую запись в коллекции и инициирует процесс генерации с указанными настройками.

2.4 Требования к веб-интерфейсу и программному интерфейсу

Платформа предоставляет программный интерфейс в стиле REST для взаимодействия с клиентской частью. Все эндпоинты возвращают данные в формате JSON и используют стандартные коды ответов протокола HTTP. Основные группы методов включают: регистрацию и аутентификацию пользователей, генерацию новых фракталов, получение списка сохранённых изображений, загрузку деталей конкретного фрактала, обновление метаданных, управление тегами и удаление записей.

Все входящие данные проходят обязательную валидацию на уровне схем, описанных с использованием библиотеки Pydantic. Система проверяет типы параметров, диапазоны допустимых значений и наличие обязательных полей. При ошибке валидации пользователь получает понятное сообщение с указанием проблемного параметра и требований к его значению. Это предотвращает падение сервера из-за некорректных запросов и улучшает пользовательский опыт.

Интерфейс включает форму генерации с динамическим отображением параметров в зависимости от выбранного типа фрактала, страницу коллекции с сеточным отображением изображений и модальные окна для редактирования метаданных. Поддержка тёмной темы и адаптивная вёрстка обеспечивают комфортное использование платформы на устройствах с различными характеристиками экрана.

2.5 Требования к хранению данных

Для хранения метаданных фракталов используется реляционная база данных SQLite, которая не требует отдельной установки и удобна для разработки в условиях ограниченных ресурсов. Схема базы данных включает таблицы для хранения информации о пользователях, фракталах и тегах, с соответствующими внешними ключами для обеспечения целостности ссылок. Для работы с базой данных применяется объектно-реляционное отображение через библиотеку SQLAlchemy, что позволяет описывать структуру таблиц в виде классов Python и выполнять запросы с использованием объектно-ориентированного синтаксиса.

Изображения фракталов сохраняются в файловую систему в отдельной директории. Имя файла формируется на основе уникального идентификатора

фрактала и пользователя, что гарантирует отсутствие коллизий даже при одновременной генерации несколькими пользователями. Путь к файлу хранится в базе данных, что позволяет при необходимости изменить стратегию хранения без изменения бизнес-логики приложения.

2.6 Требования к безопасности и надёжности

Реализована базовая защита от некорректных запросов: ограничение максимального размера изображения, проверка допустимых диапазонов значений параметров, экранирование пользовательского ввода. Эти меры предотвращают случайные или намеренные попытки перегрузить сервер или вызвать ошибки в процессе выполнения.

Для аутентификации пользователей применяется механизм токенов JSON Web Token, который обеспечивает состояние взаимодействия без необходимости хранения сессий на сервере. Пароли пользователей хранятся в хешированном виде с использованием алгоритма bcrypt, что защищает учётные данные в случае компрометации базы данных. Все защищённые эндпоинты требуют наличия действительного токена в заголовке запроса, что предотвращает несанкционированный доступ к данным других пользователей.

2.7 Обоснование выбора требований

Объектно-ориентированная архитектура, заложенная в требованиях, позволяет чётко разделить ответственность между компонентами системы: классы генераторов отвечают за математические вычисления, классы репозиториев – за работу с данными, сервисы приложения – за координацию бизнес-сценариев, контроллеры – за обработку веб-запросов. Такое разделение упрощает тестирование, отладку и дальнейшее расширение системы, так как каждый компонент может быть модифицирован или заменён независимо от остальных.

Выбор технологического стека обусловлен доступностью, хорошей документацией и активным сообществом разработчиков. Язык программирования Python предоставляет выразительный синтаксис и богатую стандартную библиотеку. Фреймворк FastAPI обеспечивает высокую производительность за счёт асинхронной обработки запросов и автоматическую генерацию документации. Библиотека SQLAlchemy упрощает работу с реляционными базами данных через объектно-реляционное отображение. Все используемые библиотеки устанавливаются через стандартный менеджер пакетов и не требуют сложной настройки, что важно для учебного проекта с ограниченным временем на реализацию.

Таким образом, сформулированные функциональные требования обеспечивают реализуемость проекта в установленные сроки и создают основу для демонстрации применения принципов объектно-ориентированного программирования при разработке веб-приложений.

3 АНАЛИЗ ООП В ЛИТЕРАТУРЕ

3.1 Значимость объектно-ориентированного проектирования

Современная разработка программного обеспечения характеризуется высокой степенью сложности, динамичностью требований и необходимостью поддержки систем на протяжении длительных жизненных циклов. В условиях, когда масштаб проектов многократно возрос, а команды разработки стали распределёнными и многоуровневыми, процедурные и структурные подходы к написанию кода продемонстрировали свою ограниченность. Ключевым ответом индустрии на эти вызовы стало объектно-ориентированное программирование, которое быстро трансформировалось из просто синтаксической парадигмы в полноценную методологию проектирования программных систем.

Как отмечают исследователи и практики, объектно-ориентированное программирование представляет собой не просто способ организации кода, а философскую и инженерную модель, позволяющую отображать предметную область в виде системы взаимодействующих объектов, каждый из которых инкапсулирует данные и поведение [1, с. 4]. Переход от линейного выполнения инструкций к событийно-объектной модели позволил кардинально снизить нагрузку на разработчиков, повысить модульность систем и обеспечить механизмы повторного использования компонентов. Главной целью объектно-ориентированного программирования является борьба со сложностью программных систем через абстрагирование от деталей реализации и локализацию кода и данных [1, с. 6]. Именно этот аспект делает объектно-ориентированное проектирование фундаментом для создания масштабируемых веб-приложений, к числу которых относится и разрабатываемая платформа для управления коллекциями фракталов.

Важно отметить, что объектно-ориентированное проектирование вышло далеко за рамки синтаксиса конкретных языков программирования. В современных реалиях это набор архитектурных принципов, паттернов взаимодействия и стандартов, определяющих, как компоненты системы должны быть структурированы, как они должны общаться друг с другом и как должна организовываться эволюция кодовой базы. Даже при использовании языка с динамической типизацией, где всё является объектом, именно предварительное проектирование и корректное выделение классов определяют, будет ли система поддерживаемой или превратится в «спагетти-код» [1, с. 4]. Таким образом, анализ литературы по объектно-ориентированному проектированию позволяет сформировать методологический базис для построения архитектуры веб-платформы, отвечающей требованиям современности: тестируемости, расширяемости, изоляции зависимостей и чёткому разделению ответственности.

3.2 Фундаментальные принципы ООП

Ядро объектно-ориентированного подхода составляют четыре базовых принципа: инкапсуляция, наследование, полиморфизм и абстракция.

Инкапсуляция в проектировании трактуется не просто как сокрытие данных за модификаторами доступа, а как механизм формирования чётких границ ответственности. Инкапсуляция обеспечивает управляемый доступ к закрытым членам класса, превращая класс в «чёрный ящик», с которым можно взаимодействовать только через заранее определённый интерфейс [1, с. 31]. В проектировании это означает, что каждый компонент системы должен предоставлять наружу только те операции, которые необходимы для выполнения его контракта, скрывая внутреннюю логику, алгоритмы и состояние. Это напрямую снижает связность между модулями и позволяет изменять реализацию одного компонента без каскадных правок в других.

Наследование исторически рассматривалось как механизм повторного использования кода, однако современная литература по проектированию предупреждает о рисках чрезмерного использования наследования. Наследование создаёт жёсткую связь между родительским и дочерним классами, что может затруднять рефакторинг. В Python наследование позволяет выносить общую логику в базовые классы, но подчёркивает важность соблюдения семантики «является», а не просто механического копирования атрибутов [1, с. 21]. В проектировании наследование целесообразно применять только для моделирования иерархий предметной области, где дочерние сущности действительно представляют собой специализацию родительских. Для веб-платформы это означает выделение абстрактного базового класса IFractalGenerator, от которого наследуются конкретные реализации алгоритмов генерации, обеспечивая единый интерфейс при различных математических подходах.

Полиморфизм является ключевым инструментом обеспечения расширяемости систем. Он позволяет работать с объектами разных классов через единый интерфейс, скрывая детали реализации. В статически типизированных языках полиморфизм реализуется через виртуальные методы и переопределение. В Python, благодаря динамической типизации и «утиной типизации», полиморфизм достигается естественным путём: если объект реализует необходимый набор методов, он может быть использован в соответствующем контексте [1, с. 28–29]. В проектировании полиморфизм позволяет создавать гибкие конвейеры обработки данных, где компоненты могут подставляться или заменяться без изменения клиентского кода.

Абстракция в проектировании означает выделение существенных характеристик объектов и игнорирование несущественных деталей. Она реализуется через абстрактные классы и интерфейсы. Абстрактные классы предназначены исключительно для наследования и не могут быть инстанциированы, а их методы должны быть реализованы в производных классах [1, с. 26]. В архитектуре веб-приложений абстракция используется для

определения слоёв: доменный слой оперирует бизнес-сущностями без привязки к базе данных или фреймворку, что соответствует принципам чистой архитектуры.

3.3 Принципы чистой архитектуры и управление зависимостями

Развитие объектно-ориентированного проектирования привело к формализации набора эмпирических правил, известных как принципы чистой архитектуры. Принципы разделения ответственности и инверсии зависимостей рассматриваются как обязательный базис для построения архитектуры веб-приложений [5, с. 38]. Чистая архитектура основывается на разделении системы на слои с чётко определёнными обязанностями, что позволяет эффективно управлять зависимостями и минимизировать их влияние на изменения [5, с. 37].

Многослойная архитектура является наиболее распространённой структурой. Чистая архитектура предполагает стратификацию системы на уровни: уровень представления, уровень приложения, доменный уровень и инфраструктурный уровень [5, с. 39]. Правило направленности зависимостей к центру системы означает, что внешний уровень может зависеть от внутреннего, но не наоборот. Объектно-ориентированное программирование обеспечивает реализацию этого правила через интерфейсы и абстрактные классы, определяемые в доменном слое, которые затем реализуются в инфраструктурном. Это позволяет заменить SQLite на PostgreSQL или изменить REST-контроллеры, не затрагивая бизнес-логику. В разрабатываемой платформе слои чётко разделены: доменный слой содержит сущности Fractal и Value Objects, слой приложения реализует сценарии использования, инфраструктурный слой предоставляет конкретные реализации репозиториев и генераторов, а уровень представления обрабатывает HTTP-запросы через FastAPI.

Эффективность управления зависимостями в данной архитектуре достигается через имплементацию принципа инверсии зависимостей. Квантификация данного принципа проявляется в организации взаимодействий между архитектурными слоями. В частности, элементы доменного слоя не должны содержать прямых зависимостей от компонентов уровня данных [5, с. 39]. Практическое применение данного подхода реализуется через взаимодействие компонентов доменного слоя с абстракциями, определёнными в том же слое, в то время как конкретная реализация этих абстракций предоставляется на уровне инфраструктуры. Исследования подтверждают эффективность такой организации для обеспечения гибкости и тестируемости системы.

3.4 Применение паттернов проектирования

Паттерны проектирования представляют типовые решения

архитектурных задач. В отличие от общей архитектуры системы, их имплементация фокусируется на микроуровне реализации кода [5, с. 40]. В контексте разрабатываемой платформы наиболее релевантными являются следующие паттерны.

Паттерн Strategy инкапсулирует семейство алгоритмов и делает их взаимозаменяемыми. Это прямой ответ на требование поддержки разных типов фракталов. Алгоритм рендеринга множества Мандельброта, множества Жюлиа, фрактала «Корабль» и фрактального пламени реализуются как разные стратегии, выбираемые в момент выполнения через фабрику. Каждый генератор наследует абстрактный интерфейс IFractalGenerator и реализует метод generate, что позволяет использовать полиморфизм для обработки результатов без знания конкретного типа алгоритма. Полиморфизм без наследования в форме «утиной типизации» доступен в Python благодаря его динамической системе типизации, когда выполнение нужной операции определяется обращением к соответствующему методу [1, с. 29].

Паттерн Template Method определяет скелет алгоритма в базовом классе, оставляя реализацию отдельных шагов дочерним классам. Класс BaseFractalGenerator реализует общий алгоритм генерации: валидация входных параметров, вычисление значений итерационного процесса и преобразование результатов в цветовое пространство. Конкретные генераторы переопределяют только метод _compute, содержащий математическую логику соответствующего фрактала, что обеспечивает повторное использование кода валидации и визуализации. Такой подход соответствует принципу открытости/закрытости, позволяя добавлять новые типы фракталов без изменения существующего кода базового класса.

Паттерн Factory Method инкапсулирует логику создания объектов, возвращая экземпляры через общий интерфейс. Класс FractalGeneratorFactory создаёт конкретный генератор на основе строкового идентификатора типа фрактала, что позволяет клиентскому коду работать только с абстракцией IFractalGenerator. Соблюдение принципа открытости/закрытости позволяет добавлять новую функциональность без изменения уже протестированного кода ядра системы [5, с. 38]. В платформе это реализуется через регистрацию новых генераторов в фабричном классе, что полностью соответствует требованиям расширяемости.

Паттерн Observer определяет зависимость «один-ко-многим» между объектами. В асинхронных платформах он используется для уведомления клиентов о завершении фоновой генерации. Класс AsyncFractalWorker наследует Subject и уведомляет подписчиков об изменениях статуса задачи, а TaskStatusTracker реализует интерфейс Observer для обновления состояния. Такой подход позволяет развязать производителя событий и потребителей, обеспечивая возможность добавления новых каналов уведомлений без изменения логики воркера.

Паттерн Decorator динамически добавляет объектам новые обязанности. В платформе он применяется для пост-обработки изображений: классы

GammaCorrectionProcessor, NormalizationProcessor и WatermarkProcessor реализуют интерфейс IImageProcessor и могут комбинироваться в цепочку обработки. Композиция позволяет реализовывать так называемый композиционный подход, когда класс-контейнер включает в себя вызовы других классов [1, с. 30]. Это обеспечивает гибкое комбинирование эффектов без наследования и соответствует принципу единственной ответственности.

Паттерн Repository абстрагирует доступ к данным, предоставляя коллекционно-ориентированный интерфейс для работы с сущностями. Класс SQLAlchemyFractalRepository реализует интерфейс IFractalRepository, инкапсулируя логику работы с базой данных через SQLAlchemy. Именно принцип инверсии зависимостей позволяет изолировать бизнес-логику от фреймворков, баз данных и внешних API [5, с. 39–40]. В платформе это реализуется через механизм внедрения зависимостей FastAPI, где контроллеры зависят от интерфейсов репозиториев, а конкретная реализация подставляется на этапе конфигурации.

Паттерн Specification инкапсулирует бизнес-правила в переиспользуемые объекты. Класс FractalValidator проверяет корректность параметров генерации, обеспечивая централизованную валидацию без дублирования кода в сущностях. Такой подход позволяет легко изменять правила валидации без модификации бизнес-сценариев и соответствует принципу разделения ответственности.

Паттерн Facade предоставляет простой интерфейс к сложной подсистеме. Класс StatsFacade скрывает сложность агрегирующих запросов к базе данных, предоставляя единый метод получения статистики пользователя. Чёткое разграничение логики и данных позволяет разработчикам легко тестировать, поддерживать и адаптировать приложение к новым требованиям [5, с. 37]. В платформе это упрощает клиентский код контроллеров и обеспечивает возможность кэширования статистики без изменения внешнего интерфейса.

Паттерн Composite позволяет единообразно работать с одиночными объектами и их композициями. В платформе он применяется для системы тегов: фрактал может иметь произвольное количество меток, которые хранятся в отдельной таблице и управляются через единый интерфейс. Это обеспечивает гибкую организацию коллекции без изменения структуры сущности и поддерживает фильтрацию по произвольным комбинациям тегов.

Паттерн Builder упрощает создание сложных объектов с множеством параметров. Класс FractalBuilder предоставляет цепочечный интерфейс для пошагового конструирования параметров фрактала, обеспечивая валидацию на каждом этапе и установку значений по умолчанию. Задорожный отмечает, что в ООП очень важно предварительное проектирование, и выделение ключевых свойств и методов объектов является критическим этапом разработки [1, с. 33]. Builder инкапсулирует логику создания объектов, упрощая клиентский код и предотвращая ошибки при инициализации.

Value Object представляет неизменяемый объект, определяемый своими

атрибутами, а не идентичностью. Классы FractalParams и ImageMetadata реализованы как frozen dataclasses, что обеспечивает безопасность потоков и предсказуемость поведения. Использование Value Objects позволяет типизировать параметры генерации, обеспечивая самодокументируемость кода и автоматическую валидацию.

3.5 Особенности реализации ООП в языке Python

Особенности языка Python, выбранного в качестве основы для разрабатываемой платформы, обладают уникальными характеристиками в реализации объектно-ориентированной парадигмы, которые необходимо учитывать при проектировании архитектуры. В Python всё является объектом, включая функции, классы и модули, а динамическая типизация предоставляет гибкость, но требует дисциплины проектирования [1, с. 4–5].

Отсутствие жёсткой инкапсуляции на уровне языка компенсируется соглашениями об именовании. Атрибуты с одним подчёркиванием считаются защищёнными, с двумя – приватными. В проектировании это означает, что границы ответственности поддерживаются не компилятором, а архитектурными договорённостями и код-ревью. Это требует от разработчика большей осознанности при проектировании интерфейсов классов.

Утиная типизация позволяет реализовать полиморфизм без явного наследования от общего интерфейса. Если класс реализует методы generate и validate, он может использоваться в контексте генератора фракталов. Это ускоряет разработку, но затрудняет статический анализ. Для компенсации в современной Python-разработке используются типизационные подсказки и инструменты статической проверки. В FastAPI это интегрировано на уровне Pydantic-моделей, которые автоматически валидируют данные и генерируют схемы документации.

Механизм декораторов позволяет расширять поведение классов и методов без изменения их кода. Декораторы в Python являются мощным инструментом аспектно-ориентированного программирования, встроенного в ООП-контекст. В платформе они используются для валидации запросов, обработки исключений и логирования, что позволяет выносить сквозную функциональность в отдельные модули, сохраняя бизнес-логику чистой и тестируемой.

3.6 Архитектурные решения для разрабатываемой платформы

Анализ литературы позволяет сформулировать конкретные архитектурные решения для платформы управления коллекциями фракталов, основанные на объектно-ориентированном проектировании. Доменный слой содержит абстрактные классы и интерфейсы, определяющие контракты генерации, хранения и обработки изображений. Эти интерфейсы не зависят от внешних библиотек. Бизнес-сущности реализованы как простые классы

данных или модели Pydantic, инкапсулирующие параметры генерации и метаданные.

Слой приложения реализует сценарии использования, каждый из которых получает зависимости через внедрение, соблюдая принципы единственной ответственности и инверсии зависимостей. Это позволяет тестировать каждый сценарий изолированно, подставляя заглушки зависимостей. Инфраструктурный слой содержит конкретные реализации интерфейсов: генераторы фракталов, репозиторий на SQLAlchemy, обработчики изображений. Фабричный паттерн отвечает за выбор конкретной реализации на основе входных параметров, обеспечивая расширяемость.

Уровень представления состоит из маршрутизаторов FastAPI и схем запросов и ответов. Контроллеры зависят только от интерфейсов слоя приложения, не обращаясь напрямую к базе данных или файловой системе. Валидация входных данных осуществляется на уровне схем, что соответствует требованию надёжности и защиты от некорректных запросов.

Управление состоянием и асинхронность реализованы через паттерны Command и Observer. Задача генерации обрабатывается в фоновом потоке, а статус отслеживается через polling. Это предотвращает блокировку основного потока обработки запросов, что критично для вычислительно сложных операций. Применение этих подходов особенно важно в условиях постоянно меняющихся технологий и быстро растущих требований к веб-приложениям, позволяя системам оставаться актуальными и производительными на протяжении длительного времени [5, с. 37].

Такое проектирование обеспечивает расширяемость, поддерживаемость, типобезопасность и независимость от технологий. Теоретический фундамент, изложенный в данном разделе, непосредственно переходит в практическую плоскость проектирования системы.

Особенности языка Python не отменяют необходимости в дисциплинированном проектировании. Напротив, они требуют от разработчика более глубокого понимания контрактов, интерфейсов и границ ответственности, так как компилятор не может принудительно обеспечить соблюдение архитектурных правил. Интеграция Python с современными фреймворками предоставляет готовые механизмы для реализации принципов объектно-ориентированного проектирования: внедрение зависимостей для инверсии зависимостей, модели для валидации и инкапсуляции данных, асинхронность для эффективного управления ресурсами.

Для разрабатываемой платформы управления коллекциями фракталов выбранный архитектурный подход, основанный на чистой архитектуре, разделении слоёв, использовании фабрик, стратегий и репозиториев, является оптимальным. Он позволяет реализовать требуемый функционал в виде системы слабосвязанных, тестируемых и расширяемых компонентов. Теоретический фундамент, изложенный в данном разделе, непосредственно переходит в практическую плоскость проектирования системы.

4 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

Проектирование веб-платформы для управления коллекциями фракталов осуществлялось с учётом требований объектно-ориентированного программирования, современных стандартов разработки веб-приложений и специфики вычислительных задач, связанных с генерацией фрактальной графики. В данном разделе рассматриваются выбранные технологии, архитектурные решения, применение паттернов проектирования, а также теоретические основы и практическая реализация алгоритмов генерации фракталов.

4.1 Выбор технологического стека

Для реализации серверной части платформы был выбран язык программирования Python версии 3.13. Данный выбор обусловлен несколькими факторами. Во-первых, Python предоставляет выразительный и лаконичный синтаксис, что способствует повышению читаемости кода и ускорению разработки. Во-вторых, язык обладает богатой экосистемой библиотек для научных вычислений, обработки изображений и веб-разработки. В-третьих, в Python объектно-ориентированное программирование играет ключевую роль, поскольку практически все сущности языка представляют собой объекты, что делает его идеальным выбором для проекта, демонстрирующего принципы ООП [1, с. 4].

В качестве веб-фреймворка для построения REST API использован FastAPI версии 0.115. Данный фреймворк обеспечивает высокую производительность за счёт асинхронной обработки запросов, автоматическую генерацию интерактивной документации в формате OpenAPI и строгую типизацию данных через аннотации типов Python и библиотеку Pydantic. Современные фреймворки, поддерживающие механизм внедрения зависимостей и декларативную конфигурацию, существенно упрощают реализацию принципов чистой архитектуры и инверсии зависимостей [5, с. 39].

Для работы с реляционной базой данных применена библиотека SQLAlchemy версии 2.0, предоставляющая объектно-реляционное отображение и поддержку транзакций. В качестве СУБД на этапе разработки выбрана SQLite, не требующая отдельной установки и удобная для прототипирования. Архитектура репозиториев спроектирована таким образом, что замена SQLite на PostgreSQL или другую СУБД потребует изменения только реализации интерфейса, не затрагивая бизнес-логику приложения.

Для выполнения вычислительно сложных операций генерации фракталов использована библиотека NumPy, обеспечивающая векторизованные операции над многомерными массивами. Обработка и сохранение изображений осуществляется через библиотеку Pillow. Для

аутентификации пользователей применены библиотека python-jose для работы с JWT-токенами и passlib с алгоритмом bcrypt для безопасного хеширования паролей.

Фронтенд-часть платформы реализована с использованием стандартных технологий веб-разработки: HTML, CSS и JavaScript. Отказ от тяжёлых фреймворков в пользу нативных технологий обусловлен стремлением к минимальному размеру клиентского кода, высокой производительности и простоте развёртывания.

4.2 Архитектурное проектирование системы

Архитектура платформы построена на принципах чистой архитектуры, предполагающей стратификацию системы на функциональные уровни с чётко определёнными границами ответственности. Чистая архитектура основывается на разделении системы на слои, что позволяет эффективно управлять зависимостями и минимизировать их влияние на изменения [5, с. 37]. В разрабатываемой системе выделены четыре основных слоя: доменный слой, слой приложения, инфраструктурный слой и слой представления.

Доменный слой содержит бизнес-сущности и интерфейсы, не зависящие от внешних технологий. Ключевыми элементами данного слоя являются классы Fractal и User, представляющие основные объекты предметной области, а также value-объекты FractalParams и ImageMetadata, инкапсулирующие параметры генерации и метаданные изображений. Интерфейсы IFractalGenerator, IFractalRepository и IImageStorage определяют контракты для генерации фракталов, работы с данными и хранения файлов соответственно. Согласно принципу инверсии зависимостей, доменный слой не содержит ссылок на внешние библиотеки, что обеспечивает его независимость и тестируемость [5, с. 39].

Слой приложения реализует сценарии использования системы через классы use-case: GenerateFractalUseCase, GetCollectionUseCase, UpdateFractalUseCase и другие. Каждый use-case получает зависимости через внедрение, соблюдая принцип единственной ответственности. Это позволяет тестировать каждый сценарий изолированно, подставляя заглушки зависимостей. Валидация входных данных осуществляется через сервис FractalValidator, реализующий паттерн Specification для объектной проверки параметров генерации.

Инфраструктурный слой содержит конкретные реализации интерфейсов доменного слоя. Генераторы фракталов (MandelbrotGenerator, JuliaGenerator, BurningShipGenerator, IFSFractalGenerator) наследуют абстрактный класс BaseFractalGenerator и реализуют математические алгоритмы вычислений. Репозиторий SQLAlchemyFractalRepository обеспечивает работу с базой данных через ORM. Обработчики изображений (GammaCorrectionProcessor, NormalizationProcessor) реализуют паттерн Decorator для гибкой пост-обработки. Фабрика FractalGeneratorFactory инкапсулирует логику выбора

конкретного генератора на основе входного параметра.

Слой представления состоит из маршрутизаторов FastAPI и схем запросов и ответов на базе Pydantic. Контроллеры зависят только от интерфейсов слоя приложения, не обращаясь напрямую к базе данных или файловой системе. Валидация входных данных осуществляется на уровне схем, что соответствует требованию надёжности и защиты от некорректных запросов. Автоматическая генерация документации OpenAPI упрощает процесс отладки и интеграции с внешними системами.

Правило направленности зависимостей к центру системы обеспечивает, что внешний слой может зависеть от внутреннего, но не наоборот. Это позволяет заменять реализации инфраструктурного слоя без изменения бизнес-логики. Например, замена SQLite на PostgreSQL потребует модификации только класса SQLAlchemyFractalRepository, не затрагивая use-case или доменные сущности.

4.3 Применение паттернов проектирования

В процессе проектирования платформы было применено несколько классических паттернов проектирования, каждый из которых решает конкретную архитектурную задачу. Правильное применение паттернов формализует терминологию внутри команды и упрощает коммуникацию, заменяя долгие описания поведения лаконичными названиями архитектурных решений [5, с. 40].

Паттерн Strategy применён для реализации алгоритмов генерации различных типов фракталов. Абстрактный класс IFractalGenerator определяет единый интерфейс метода generate, который реализуют конкретные классы генераторов. Это позволяет клиентскому коду работать с любым генератором через общий интерфейс, не завися от конкретной реализации. Добавление нового типа фрактала требует создания одного класса и регистрации в фабрике, что соответствует принципу открытости/закрытости [5, с. 38].

Паттерн Template Method реализован в базовом классе BaseFractalGenerator, который определяет скелет алгоритма генерации: валидация параметров, вычисление значений итерационного процесса и преобразование результатов в цветовое пространство. Конкретные генераторы переопределяют только абстрактный метод _compute, содержащий математическую логику соответствующего фрактала. Такой подход обеспечивает повторное использование кода валидации и визуализации, устраняя дублирование.

Паттерн Factory Method инкапсулирует логику создания объектов генераторов в классе FractalGeneratorFactory. Метод create принимает строковый идентификатор типа фрактала и возвращает соответствующий экземпляр через общий интерфейс. Клиентский код не зависит от конкретных классов генераторов, что упрощает тестирование и расширяемость системы. Фабрики позволяют выносить логику создания объектов, делая код более

читаемым и поддерживаемым [1, с. 30].

Паттерн Observer применён для реализации асинхронных уведомлений о завершении генерации. Класс AsyncFractalWorker наследует Subject и уведомляет подписчиков об изменениях статуса задачи через метод notify. Класс TaskStatusTracker реализует интерфейс Observer и обновляет внутреннее состояние при получении событий. Такой подход позволяет развязать производителя событий и потребителей, обеспечивая возможность добавления новых каналов уведомлений без изменения логики воркера.

Паттерн Decorator используется для пост-обработки изображений. Классы GammaCorrectionProcessor, NormalizationProcessor и WatermarkProcessor реализуют интерфейс IImageProcessor и могут комбинироваться в цепочку обработки. Каждый декоратор принимает опциональный параметр wrapped, представляющий следующий элемент цепочки. Это обеспечивает гибкое комбинирование эффектов без наследования и соответствует принципу единственной ответственности.

Паттерн Repository абстрагирует доступ к данным, предоставляя коллекционно-ориентированный интерфейс для работы с сущностями. Класс SQLAlchemyFractalRepository реализует интерфейс IFractalRepository, инкапсулируя логику работы с базой данных через SQLAlchemy. Согласно принципу инверсии зависимостей, бизнес-логика зависит только от абстракций, что позволяет заменять реализацию репозитория без изменения use-case [5, с. 39–40].

Паттерн Specification инкапсулирует бизнес-правила валидации в классе FractalValidator. Метод validate проверяет корректность параметров генерации, обеспечивая централизованную проверку без дублирования кода в сущностях. Такой подход позволяет легко изменять правила валидации без модификации бизнес-сценариев.

Паттерн Facade реализован в классе StatsFacade, который предоставляет простой интерфейс для получения агрегированной статистики пользователя. Метод get_user_stats скрывает сложность выполнения нескольких запросов к базе данных, возвращая готовый объект UserStats. Это упрощает клиентский код контроллеров и обеспечивает возможность кэширования статистики без изменения внешнего интерфейса.

Паттерн Composite применён для системы тегов. Фрактал может иметь произвольное количество меток, которые хранятся в отдельной таблице и управляются через единый интерфейс. Это обеспечивает гибкую организацию коллекции без изменения структуры сущности и поддерживает фильтрацию по произвольным комбинациям тегов.

Паттерн Builder упрощает создание сложных объектов с множеством параметров. Класс FractalBuilder предоставляет цепочечный интерфейс для пошагового конструирования параметров фрактала, обеспечивая валидацию на каждом этапе и установку значений по умолчанию. Композиция позволяет реализовывать подход, когда класс-контейнер включает в себя вызовы других классов, что упрощает создание объектов со сложной структурой [1, с. 30].

Value Object представляет неизменяемый объект, определяемый своими атрибутами, а не идентичностью. Классы FractalParams и ImageMetadata реализованы как frozen dataclasses, что обеспечивает безопасность потоков и предсказуемость поведения. Использование Value Objects позволяет типизировать параметры генерации, обеспечивая самодокументируемость кода и автоматическую валидацию.

4.4 Теоретические основы генерации фракталов

Фракталы представляют собой геометрические объекты, обладающие свойством самоподобия: их структура повторяется при изменении масштаба. Фрактальный анализ является мощным инструментом для исследования сложных систем, обладающих нерегулярной структурой и масштабной инвариантностью [6, с. 15]. В разрабатываемой платформе реализована генерация четырёх типов фракталов: множества Мандельброта, множества Жюлиа, фрактала «Корабль» и фрактального пламени на основе системы итерированных функций.

Множество Мандельброта определяется как множество точек комплексной плоскости, для которых последовательность, заданная рекуррентным соотношением, не стремится к бесконечности. Визуализация множества Мандельброта позволяет наблюдать бесконечное разнообразие фрактальных структур при увеличении масштаба [6, с. 42].

Параметры генерации множества Мандельброта включают координаты центра области просмотра, масштаб отображения, максимальное количество итераций и размеры изображения в пикселях. Координаты центра определяют область комплексной плоскости, которая будет визуализирована. Масштаб управляет уровнем увеличения: меньшие значения соответствуют более глубокому зуму и большей детализации границ. Количество итераций влияет на точность определения принадлежности точки множеству: большее значение обеспечивает плавные цветовые переходы, но увеличивает время вычислений.

Множество Жюлиа определяется аналогичным рекуррентным соотношением, однако константа с*c* фиксируется для всей плоскости. Форма множества Жюлиа существенно зависит от выбора параметра c: при значениях внутри множества Мандельброта получается связное множество, при значениях снаружи – несвязная «пыль». Множества Жюлиа демонстрируют богатое разнообразие форм, от симметричных структур до хаотичных фрактальных облаков [6, с. 48].

Фрактал «Корабль» приводит к появлению характерной асимметричной формы, напоминающей парусное судно. При увеличении масштаба вблизи границ наблюдаются повторяющиеся «парусообразные» структуры. Параметры генерации аналогичны множеству Мандельброта: центр, масштаб, количество итераций и размеры изображения.

Фрактальное пламя генерируется на основе системы итерированных

функций. Алгоритм использует метод «chaos game»: начиная с произвольной точки, на каждом шаге случайно выбирается одно из аффинных преобразований, которое применяется к текущей точке. Аффинное преобразование задаётся матрицей коэффициентов и вектором сдвига. После применения аффинного преобразования может быть применена нелинейная вариационная функция, такая как sinusoidal, spherical, swirl, heart и другие. Использование нелинейных вариаций позволяет получать художественные эффекты, недоступные при применении только линейных преобразований [6, с. 73].

Параметры генерации фрактального пламени включают название пресета, наименование вариационной функции, количество сэмплов, начальное значение генератора случайных чисел и, при необходимости, пользовательские аффинные преобразования с весами. Пресеты определяют набор аффинных преобразований и область просмотра. Вариационная функция управляет нелинейным искажением точек. Количество сэмплов влияет на плотность изображения: большее значение обеспечивает плавные градиенты, но увеличивает время вычислений.

Визуализация фракталов осуществляется через преобразование значений итерационного процесса в цветовое пространство. Для итерационных фракталов применяется косинусная палитра с фазовым сдвигом, обеспечивающая плавные переходы между цветами. Для фрактального пламени используется гистограммная раскраска: цвет пикселя определяется частотой попадания точек в соответствующую область, что создаёт эффект свечения и глубины.

4.5 Реализация алгоритмов генерации

Реализация алгоритмов генерации фракталов осуществлена в виде иерархии классов, наследующих абстрактный базовый класс BaseFractalGenerator. Данный класс реализует шаблонный метод generate, определяющий общий алгоритм обработки: валидация параметров, вычисление значений итерационного процесса и преобразование результатов в цветовое пространство. Конкретные генераторы переопределяют только абстрактный метод _compute, содержащий математическую логику соответствующего фрактала.

Класс MandelbrotGenerator реализует алгоритм вычисления множества Мандельброта. Метод _compute создаёт сетку комплексных чисел, соответствующую области просмотра, и для каждой точки выполняет итерационный процесс согласно рекуррентному соотношению. Для оптимизации вычислений используется векторизация через библиотеку NumPy: операции над массивами выполняются одновременно для всех точек, что существенно ускоряет генерацию. Маска активных точек позволяет прекращать итерации для точек, уже вышедших за пределы множества, что дополнительно повышает производительность.

Класс JuliaGenerator реализует аналогичный алгоритм для множества Жюлиа, с той разницей, что константа фиксируется на основе параметров генерации. Реализация также использует векторизованные операции и маску активных точек для оптимизации вычислений.

Класс BurningShipGenerator отличается применением модуля к вещественной и мнимой части перед возведением в квадрат. В реализации это достигается через раздельную обработку вещественной и мнимой компонент массива комплексных чисел.

Класс IFSFractalGenerator реализует алгоритм генерации фрактального пламени. Метод _compute выполняет метод «chaos game»: начиная с точки (0,0), на каждом шаге случайно выбирается аффинное преобразование из набора, определённого пресетом или пользовательской конфигурацией. После применения аффинного преобразования может быть применена нелинейная вариационная функция. Координаты полученной точки преобразуются в координаты пикселя изображения, и соответствующий счётчик гистограммы увеличивается. После завершения итераций гистограмма нормализуется через логарифмическое масштабирование, и значения преобразуются в цветовое пространство.

Для обеспечения гибкости и расширяемости все генераторы принимают параметры в виде value-объекта FractalParams или его специализированных наследников. Валидация параметров осуществляется в методе post_init value-объектов, что гарантирует целостность состояния до начала вычислений.

4.6 Проектирование системы хранения данных

Система хранения данных спроектирована с учётом требований к целостности, производительности и возможности миграции. Метаданные фракталов хранятся в реляционной базе данных SQLite через объектно-реляционное отображение библиотеки SQLAlchemy. Схема базы данных включает таблицы users, fractals и fractal_tags, связанные внешними ключами для обеспечения ссылочной целостности.

Класс SQLAlchemyFractalRepository реализует интерфейс IFractalRepository, предоставляя методы для сохранения, получения, обновления и удаления фракталов. Метод save обеспечивает атомарное сохранение метаданных фрактала и связанных тегов в рамках одной транзакции. Метод list_by_user возвращает отсортированный список фракталов пользователя с подгрузкой тегов. Метод get_user_stats выполняет агрегирующие запросы для получения статистики: общего количества сгенерированных фракталов, среднего времени генерации и наиболее часто используемого типа фрактала.

Изображения фракталов сохраняются в файловую систему в директории media. Имя файла формируется на основе уникального идентификатора фрактала и пользователя, что гарантирует отсутствие коллизий даже при одновременной генерации несколькими пользователями. Путь к файлу

хранится в базе данных, что позволяет при необходимости изменить стратегию хранения без изменения бизнес-логики приложения.

4.7 Обеспечение асинхронности и производительности

Генерация фракталов является вычислительно сложной операцией, время выполнения которой может достигать нескольких секунд или минут в зависимости от параметров. Для предотвращения блокировки основного потока обработки HTTP-запросов реализован механизм асинхронной обработки через паттерн Active Object.

Класс AsyncFractalWorker использует ThreadPoolExecutor для выполнения задач генерации в фоновых потоках. Метод submit принимает задачу с параметрами генерации и инициирует выполнение в отдельном потоке. При старте и завершении задачи воркер уведомляет подписчиков через механизм Observer, что позволяет клиентскому приложению отслеживать статус генерации через polling-запросы.

Для оптимизации вычислений в генераторах используется векторизация через библиотеку NumPy. Операции над массивами выполняются одновременно для всех точек области просмотра, что обеспечивает существенное ускорение по сравнению с поэлементной обработкой. Маска активных точек позволяет прекращать итерации для точек, уже вышедших за пределы множества, что дополнительно повышает производительность.

Пост-обработка изображений реализована через цепочку декораторов, что позволяет гибко комбинировать алгоритмы нормализации, гамма-коррекции и добавления водяных знаков. Каждый декоратор обрабатывает изображение в формате массива NumPy с плавающей точкой в диапазоне [0, 1], что обеспечивает точность вычислений и предотвращает переполнение.

Кэширование результатов генерации реализовано через паттерн Proxy в классе FractalCacheProxy. При повторной генерации с теми же параметрами результат возвращается из кэша без выполнения вычислений. Ключ кэша формируется на основе хеша параметров генерации, что гарантирует корректность повторного использования результатов.

Таким образом, спроектированная система сочетает в себе преимущества объектно-ориентированной архитектуры, современных веб-технологий и оптимизированных алгоритмов генерации фракталов. Применение принципов чистой архитектуры, паттернов проектирования и асинхронной обработки обеспечивает расширяемость, поддерживаемость и производительность платформы, что полностью соответствует целям курсовой работы.

5 ТЕСТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

5.1 Методика и среда проведения тестирования

Тестирование разработанной веб-платформы для управления коллекциями фракталов осуществлялось в соответствии с принципами обеспечения качества программного обеспечения, изложенными в современной литературе по инженерии программных систем. Тестируемость компонентов является одним из ключевых критериев качества архитектуры, достигаемым за счёт изоляции бизнес-логики от внешних зависимостей и чёткого разделения ответственности между модулями [5, с. 38]. В рамках данного раздела описываются сценарии тестирования, методика проведения проверок, анализ поведения системы в различных условиях эксплуатации и оценка соответствия функциональным требованиям, сформулированным на этапе проектирования.

Тестирование проводилось в среде локальной разработки с использованием браузера на базе движка Chromium, сервера FastAPI, запущенного в режиме разработки с включённой перезагрузкой при изменении кода, и базы данных SQLite. Для регистрации ошибок и анализа поведения применялись инструменты разработчика браузера, логи сервера и консольные сообщения, выводимые в процессе выполнения асинхронных операций.

5.2 Тестирование механизмов аутентификации и авторизации

Первым этапом тестирования являлась проверка механизмов аутентификации и авторизации пользователей. При первоначальном посещении платформы пользователь попадает на главную страницу, где отображается форма генерации фракталов и панель управления авторизацией. В правом верхнем углу интерфейса располагается кнопка «Войти / Регистрация», при нажатии на которую открывается модальное окно с формой ввода учётных данных в соответствии с рисунком 5.1.

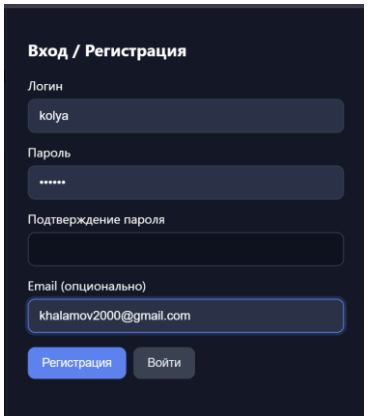


Рисунок 5.1 – Интерфейс модального окна авторизации

Форма содержит поля для ввода логина, пароля, подтверждения пароля и опционального адреса электронной почты. Система реализует валидацию пароля на стороне клиента: пароль должен содержать минимум четыре символа, хотя бы одну латинскую букву, одну цифру и один специальный символ. При вводе некорректного пароля пользователь получает информативное сообщение об ошибке без отправки запроса на сервер, что снижает нагрузку на бэкенд и улучшает пользовательский опыт.

Процесс регистрации тестируется путём ввода нового уникального логина и корректного пароля. При успешной регистрации система возвращает сообщение о создании аккаунта и автоматически переключает форму в режим входа. Пользователь вводит те же учётные данные для авторизации. Сервер проверяет корректность пароля с использованием алгоритма bcrypt, генерирует JWT-токен и возвращает его клиенту вместе с информацией о пользователе. Токен сохраняется в локальном хранилище браузера и автоматически подставляется в заголовок Authorization всех последующих запросов. После успешного входа интерфейс обновляется: кнопка авторизации скрывается, отображается имя пользователя и кнопка выхода в соответствии с рисунком 5.2.

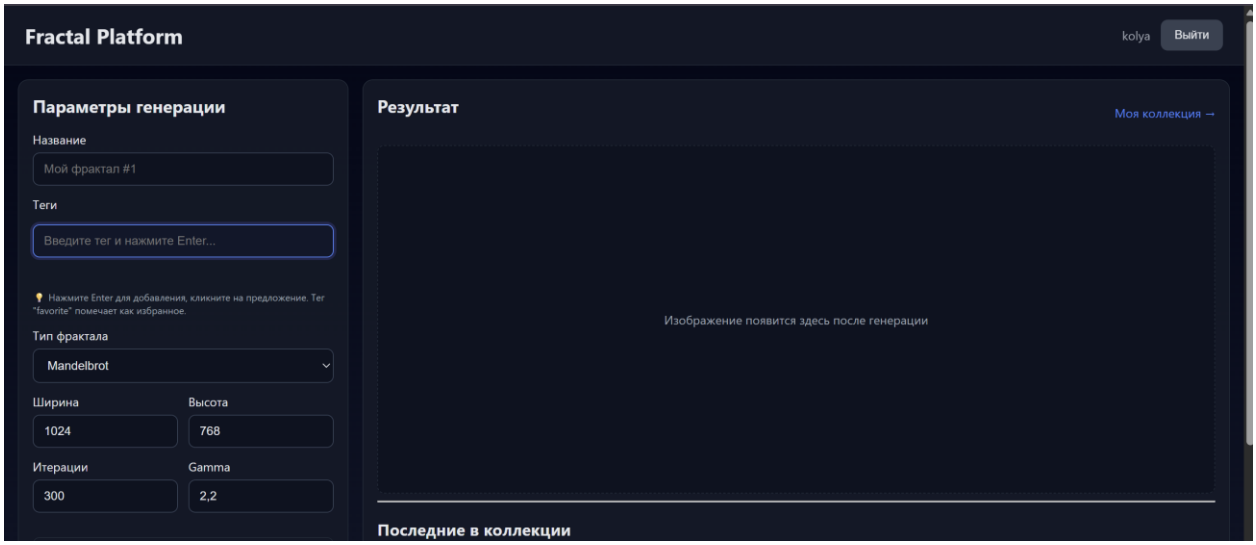


Рисунок 5.2 – Интерфейс главной страницы после авторизации

5.3 Проверка интерфейса генерации и валидации параметров

Следующим этапом тестирования являлась проверка процесса генерации фрактальных изображений. После авторизации пользователь получает доступ к форме генерации, содержащей поля для ввода названия фрактала, выбора типа алгоритма и настройки параметров в соответствии с рисунком 5.3.

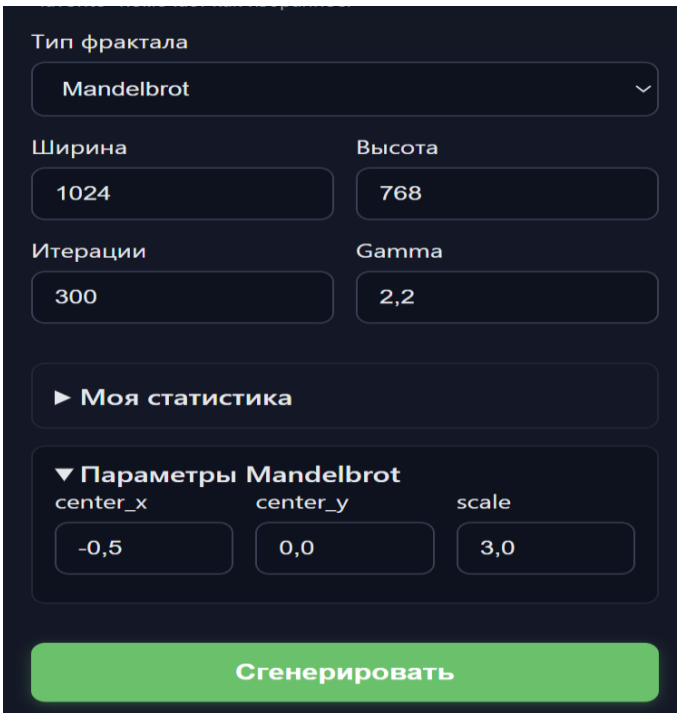


Рисунок 5.3 – Форма генерации с раскрытыми параметрами типа фрактала

При выборе типа фрактала в выпадающем списке динамически отображаются соответствующие группы полей: для множества Мандельброта – координаты центра и масштаб; для множества Жюлиа – дополнительно константы; для фрактала «Корабль» – специфичные параметры области просмотра; для фрактального пламени – выбор пресета, вариационной функции, количества сэмплов и возможность задания кастомных аффинных преобразований. Такая адаптивная форма реализована с использованием нативного JavaScript и обеспечивает интуитивное взаимодействие без перезагрузки страницы.

Параметры генерации проходят валидацию на стороне клиента и сервера. Клиентская валидация проверяет диапазоны допустимых значений: ширина и высота изображения от 256 до 2048 пикселей, количество итераций от 50 до 5000, коэффициент гамма от 0.5 до 5.0. Серверная валидация, реализованная через схемы Pydantic, дублирует эти проверки и дополнительно контролирует целостность данных. При вводе некорректных значений пользователь получает сообщение с указанием проблемного параметра. Тестирование показало, что система корректно обрабатывает граничные значения и отклоняет запросы с недопустимыми параметрами, возвращая код ответа 422 и детализированное описание ошибки.

5.4 Тестирование отображения результатов

Процесс генерации выполняется асинхронно для предотвращения блокировки основного потока обработки запросов. При нажатии кнопки «Сгенерировать» клиент отправляет POST-запрос на эндпоинт

/api/fractals/generate с параметрами в формате JSON. Сервер валидирует данные, создаёт задачу генерации, возвращает уникальный идентификатор задачи и инициирует выполнение в фоновом потоке. Клиентское приложение переходит в режим опроса статуса задачи в соответствии с рисунком 5.4.



Рисунок 5.4 – Индикатор процесса генерации с таймером

В интерфейсе отображается текущее время выполнения операции, что обеспечивает прозрачность процесса для пользователя. По завершении генерации сервер уведомляет клиента о статусе «выполнено» и идентификаторе сгенерированного фрактала.

После успешного завершения генерации изображение отображается в центральной панели результата в соответствии с рисунком 8.5.

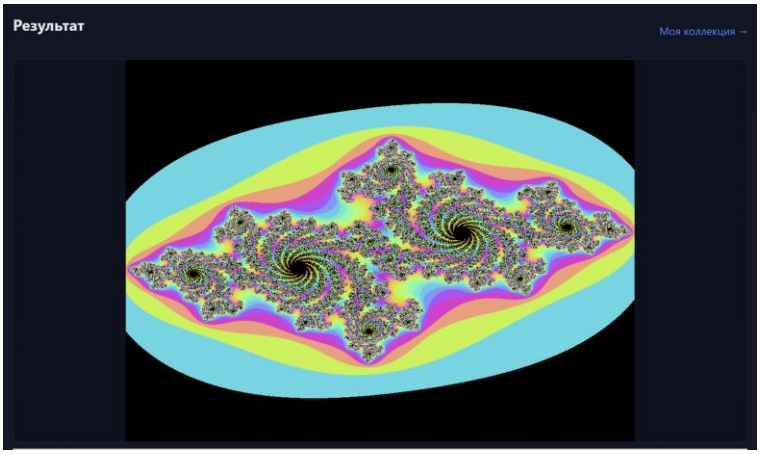


Рисунок 5.5 – Отображение сгенерированного фрактала с метаданными

Клиентское приложение добавляет параметр времени к URL для обхода кэша браузера и гарантирует актуальность отображаемого контента. Ниже основного изображения располагается мини-галерея с последними шестью фракталами из коллекции пользователя. Клик по миниатюре заменяет основное изображение, обеспечивая быстрый просмотр без перехода на отдельную страницу.

5.5 Тестирование управления коллекцией

Управление коллекцией осуществляется через отдельную страницу, доступную по ссылке «Моя коллекция» в соответствии с рисунком 5.6.

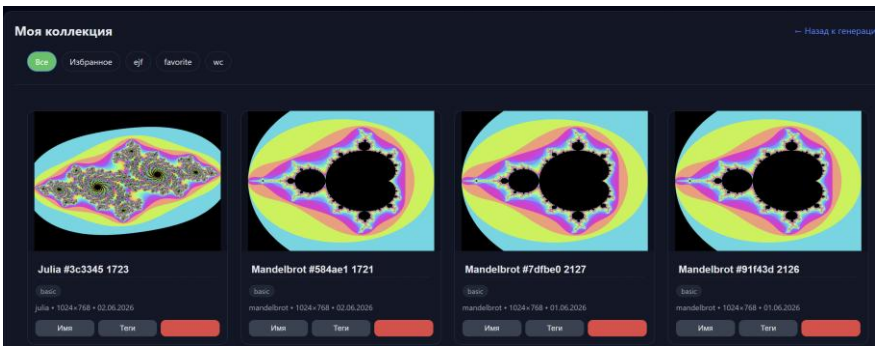


Рисунок 5.6 – Страница коллекции с сеточным отображением фракталов

Интерфейс представляет собой адаптивную сетку карточек, каждая из которых содержит превью изображения, название, тип фрактала, размеры, дату создания и список тегов. Карточки реализуют интерактивные элементы: название можно редактировать непосредственно в карточке, теги отображаются в виде кликабельных меток, кнопки действий позволяют переименовать, редактировать теги или удалить фрактал. Фильтрация коллекции по тегам осуществляется через панель в верхней части страницы, где отображаются все уникальные теги пользователя. Выбор тега мгновенно фильтрует отображаемые элементы без перезагрузки страницы.

Редактирование тегов реализовано через модальное окно, открывающееся при нажатии соответствующей кнопки на карточке в соответствии с рисунком 5.7.

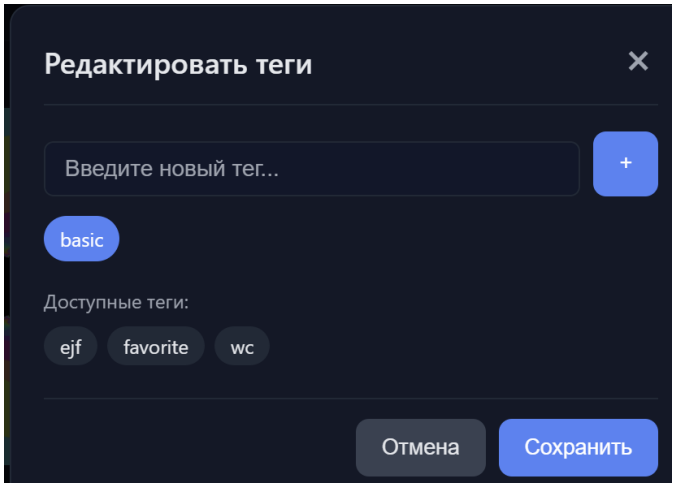


Рисунок 5.7 – Модальное окно редактирования тегов

Окно содержит список выбранных тегов с возможностью удаления, поле ввода для добавления нового тега и список доступных тегов пользователя для быстрого выбора. Система предотвращает дублирование тегов и автоматически нормализует ввод к нижнему регистру. Особый статус имеет тег «favorite», который визуально выделяется иконкой звезды и может использоваться для быстрой фильтрации избранных фракталов. Сохранение

изменений отправляет PUT-запрос на сервер, который обновляет запись в базе данных и возвращает актуальный список тегов.

5.6 Проверка удаления записей и расчёта статистикиэ

Удаление фрактала из коллекции требует подтверждения через стандартное диалоговое окно браузера для предотвращения случайной потери данных. После подтверждения клиент отправляет DELETE-запрос на сервер, который удаляет запись из базы данных и соответствующий файл изображения из файловой системы. Интерфейс автоматически обновляет список, удаляя карточку из сетки. Тестирование показало, что операция удаления выполняется атомарно: при сбое на любом этапе изменения не фиксируются, что обеспечивает целостность данных.

Функционал статистики пользователя доступен через раскрывающийся блок «Моя статистика» в форме генерации. После авторизации система загружает агрегированные данные: общее количество сгенерированных фракталов, наиболее часто используемый тип алгоритма, среднее время генерации и суммарное время всех операций. Данные рассчитываются на стороне сервера через агрегирующие SQL-запросы и возвращаются в виде структурированного JSON-объекта. Клиентское приложение форматирует значения для отображения: время переводится из миллисекунд в секунды с одним знаком после запятой, названия типов фракталов преобразуются в читаемый формат. Тестирование подтвердило корректность расчётов и актуальность отображаемой информации.

5.7 Тестирование обработки ошибок

Экспорт и импорт параметров генерации тестируется через прямые вызовы соответствующих эндпоинтов. Экспорт параметров выполняется запросом, который возвращает JSON-объект со всеми параметрами генерации, включая тип фрактала, размеры, итерации, координаты и специфичные настройки. Импорт осуществляется через POST-запрос на /api/fractals/import с параметрами в теле запроса. Система валидирует импортируемые данные, создаёт новую запись в коллекции и инициирует генерацию. Тестирование показало, что параметры экспортируются и импортируются без потери информации, что обеспечивает воспроизводимость результатов и возможность обмена настройками между пользователями.

Обработка ошибок является критическим аспектом надёжности системы. Тестирование проводилось путём намеренного ввода некорректных данных, отправки запросов с неверными токенами, имитации сетевых сбоев и перегрузки сервера. Система корректно возвращает коды ответов 400 для ошибок валидации, 401 для неавторизованных запросов, 404 для несуществующих ресурсов и 500 для внутренних ошибок сервера. Клиентское

приложение перехватывает ошибки и отображает пользователю информативные сообщения без прерывания работы интерфейса. Асинхронные уведомления реализованы через систему тостов, которые автоматически исчезают через пять секунд, не требуя ручного закрытия.

5.8 Оценка производительности

Производительность системы оценивалась при генерации изображений различных размеров и сложности. Тестирование показало, что генерация изображения 1024 на 768 пикселей с 300 итерациями занимает в среднем 1.5–3 секунды для итерационных фракталов и 3–8 секунд для фрактального пламени с 300 000 сэмплов. Асинхронная обработка позволяет серверу обрабатывать несколько запросов параллельно без блокировки основного потока. Кэширование повторных запросов с идентичными параметрами сокращает время ответа до десятков миллисекунд. Масштабируемость архитектуры подтверждается возможностью замены SQLite на PostgreSQL без изменения бизнес-логики, что обеспечивает рост производительности при увеличении нагрузки. Темная тема интерфейса снижает нагрузку на глаза при длительной работе и соответствует современным стандартам пользовательского опыта.

Результаты тестирования подтверждают соответствие разработанной платформы функциональным требованиям, сформулированным на этапе проектирования. Все основные сценарии использования реализованы и работают корректно: регистрация и аутентификация пользователей, генерация четырёх типов фракталов с настраиваемыми параметрами, асинхронная обработка задач, управление коллекцией с поддержкой тегов, экспорт и импорт параметров, отображение пользовательской статистики. Архитектурные решения, основанные на принципах объектно-ориентированного программирования и чистой архитектуры, обеспечивают тестируемость, расширяемость и поддерживаемость кодовой базы.

В процессе тестирования выявлены и устранены незначительные недостатки интерфейса: улучшена обработка ошибок сети, добавлены подсказки для полей ввода, оптимизирована загрузка мини-галереи. Эти доработки не затрагивают архитектурные решения и реализованы в рамках существующей структуры кода, что подтверждает правильность выбранного подхода к проектированию.

Таким образом, проведённое тестирование демонстрирует работоспособность и надёжность разработанной веб-платформы для управления коллекциями фракталов. Система готова к демонстрации в рамках защиты курсовой работы и может служить основой для дальнейшего развития в направлении полноценного сервиса для работы с фрактальной графикой.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана веб-платформа для управления коллекциями фракталов, реализованная на языке программирования Python с использованием фреймворка FastAPI. Основным результатом работы стало создание функционального программного продукта, демонстрирующего применение принципов объектно-ориентированного программирования для решения практической задачи генерации, визуализации и хранения фрактальных изображений. Система обеспечивает пользователю возможность регистрации и аутентификации, генерации фракталов четырёх типов с настраиваемыми параметрами, асинхронной обработки задач, управления личной коллекцией с поддержкой тегов, а также экспорта и импорта параметров генерации.

Наиболее значимым итогом работы стало глубокое усвоение принципов объектно-ориентированного проектирования и их практическое применение в архитектуре веб-приложения. В процессе разработки была реализована многослойная архитектура, соответствующая принципам чистой архитектуры. Разделение системы на доменный слой, слой приложения, инфраструктурный слой и уровень представления позволило обеспечить слабую связность компонентов и высокую тестируемость кода. Каждый слой имеет чётко определённую ответственность: доменный слой содержит бизнес-сущности и интерфейсы, не зависящие от внешних технологий; слой приложения реализует сценарии использования; инфраструктурный слой предоставляет конкретные реализации интерфейсов; уровень представления обрабатывает HTTP-запросы. Такое разделение полностью соответствует принципу инверсии зависимостей.

В рамках проекта были применены ключевые паттерны проектирования, каждый из которых решает конкретную архитектурную задачу. Паттерн Strategy был использован для реализации алгоритмов генерации различных типов фракталов. Паттерн Template Method реализован в базовом классе BaseFractalGenerator, который определяет скелет алгоритма генерации, позволяя конкретным генераторам переопределять только метод вычислений. Паттерн Factory Method инкапсулирует логику создания генераторов в классе FractalGeneratorFactory, что упрощает клиентский код и обеспечивает централизованную валидацию входных данных. Паттерн Observer применён для реализации асинхронных уведомлений о завершении генерации. Такой подход позволяет развязать производителя событий и потребителей, обеспечивая возможность добавления новых каналов уведомлений без изменения логики воркера. Паттерн Decorator используется для пост-обработки изображений. Паттерн Repository абстрагирует доступ к данным, предоставляя коллекционно-ориентированный интерфейс для работы с сущностями, что позволяет заменять реализацию репозитория без изменения бизнес-логики.

Особое внимание в работе было уделено изучению особенностей объектно-ориентированного программирования в языке Python. В процессе разработки было усвоено, что отсутствие жёсткой инкапсуляции на уровне языка компенсируется соглашениями об именовании и архитектурными договорённостями. Утиная типизация позволяет реализовать полиморфизм без явного наследования, что ускоряет разработку, но требует использования типизационных подсказок и инструментов статической проверки для обеспечения надёжности кода. Механизм декораторов в Python был применён для реализации аспектно-ориентированного программирования, что позволило вынести сквозную функциональность в отдельные модули, сохраняя бизнес-логику чистой и тестируемой.

Важным аспектом работы стало усвоение принципов тестируемости и поддерживаемости кода. Чёткое разделение ответственности между компонентами позволило проводить модульное тестирование каждого use-case изолированно, подставляя заглушки зависимостей. Использование абстрактных интерфейсов в доменном слое обеспечило возможность замены реализаций инфраструктурного слоя без изменения бизнес-логики. Строгая типизация через аннотации типов Python и библиотеку Pydantic позволила автоматизировать валидацию входных данных и генерацию документации OpenAPI.

В результате выполнения курсовой работы были достигнуты все поставленные цели: выделены общие свойства различных типов фракталов и построена иерархия классов, спроектирована объектная модель предметной области, разработан REST API на фреймворке FastAPI, реализован механизм хранения данных в базе данных, проведено тестирование работоспособности платформы. Разработанная система готова к демонстрации и может служить основой для дальнейшего развития в направлении полноценного сервиса для работы с фрактальной графикой.

Главным личным результатом работы стало формирование целостного понимания объектно-ориентированного проектирования как инженерной дисциплины, а не просто синтаксической парадигмы. Было усвоено, что правильная архитектура системы определяется не количеством классов, а качеством разделения ответственности, чёткостью интерфейсов и соблюдением принципов слабой связности. Применение паттернов проектирования перестало быть абстрактным знанием и превратилось в практический инструмент решения архитектурных задач. Полученный опыт будет полезен при разработке сложных программных систем в будущей профессиональной деятельности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Задорожный С. С., Фадеев Е. П. Объектно-ориентированное программирование на языке Python / С. С. Задорожный, Е. П. Фадеев. – Москва : Физический факультет МГУ им. М. В. Ломоносова, 2022. – 52 с.

[2] Володько Л. П., Николаенко В. Л., Николаенко Д. В. Объектно-ориентированное программирование : учебное пособие / Л. П. Володько, В. Л. Николаенко, Д. В. Николаенко. – Пинск : ПолесГУ, 2020. – 164 с.

[3] Сузи Р. А. Язык программирования Python : учеб. пособие / Р. А. Сузи. – Москва : Бином-Пресс, 2006. – 206 с.

[4] Поташов А. В. Python для серверной части высоконагруженных веб-приложений / А. В. Поташов // Научный аспект. – 2024. – Т. 23. – № 7. – С. 2767.

[5] Калашников Н. А. Применение архитектурных паттернов в построении чистой архитектуры веб-приложений / Н. А. Калашников // Universum: технические науки. – 2024. – Т. 1. – № 12 (129). – С. 37–42.

[6] Латыпова Н. В. Фрактальный анализ : учеб. пособие / Н. В. Латыпова. – Ижевск : Издательский центр «Удмуртский университет», 2020. – 121 с.

[7] Очков В. Ф., Тихонов А. И. Узоры и фракталы на Python / В. Ф. Очков, А. И. Тихонов // Cloud of Science. – 2020. – Т. 7. – № 4. – С. 764–789.

[8] Lefler S. Fractals and the chaos game / S. Lefler // MAT Exam Expository Papers. – 2006. – С. 24.

[9] Ильичев В. Ю. Применение методов аффинного преобразования матриц значений пикселей растровых изображений / В. Ю. Ильичев // Сложные системы. – 2021. – № 3. – С. 51–61.

[10] Wu X. FastAPI as a backend framework : bachelor thesis / X. Wu. – Tampere : Tampere University, 2024. – 25 с.

[11] Beaulieu A. Learning SQL: master SQL fundamentals / A. Beaulieu. – Sebastopol : O'Reilly Media, 2009. – 337 с.

[12] Стандарт предприятия – Минск : БГУИР, 2024. – 178 с.

[13] Бадд Т. Объектно-ориентированное программирование / Т. Бадд. – 1997.

[14] Python Documentation / Python [Электронный ресурс] – https://docs.python.org/3/.

[15] FastAPI Documentation / FastAPI [Электронный ресурс] – https://fastapi.tiangolo.com/.